

Test Plan — RAG Document QA System

1. Overview

This test plan defines the strategy, scope, environments, test cases, and acceptance criteria for the **RAG Document QA System**.

The application allows users to:

- upload PDF , DOCX , or TXT documents,
- ingest and chunk the document text,
- generate embeddings,
- store/retrieve chunks using FAISS or PostgreSQL + pgvector ,
- ask questions about the uploaded document through FastAPI and Streamlit ,
- receive answers with **clear section references**.

2. Objectives

The testing objectives are to verify that the application:

1. correctly accepts supported document uploads,
2. extracts and chunks document text accurately,
3. stores embeddings and chunks in the configured vector backend,
4. returns relevant answers to user questions,
5. includes **clear section references** in answers,
6. avoids repeated/stale answers across uploads,
7. behaves correctly when the LLM provider is unavailable,
8. remains stable across API, UI, and database workflows.

3. Scope

In Scope

- FastAPI endpoints: /upload , /ask
- Streamlit UI workflow
- Text extraction for TXT , PDF , and DOCX
- Chunking logic and section labeling
- Embedding generation using sentence-transformers
- Vector storage using:
 - faiss
 - pgvector
 - hybrid
- PostgreSQL persistence validation
- Answer formatting and section citations
- Error handling and fallback behavior
- Automated tests using pytest

Out of Scope

- Production-scale load balancing
- Multi-user authentication/authorization
- Cloud deployment infrastructure
- Billing/quota management for third-party LLMs
- Browser compatibility matrix beyond local validation

4. Application Under Test

Main Components

Component	File	Purpose
API layer	app/main.py	Upload and ask endpoints
Ingestion	app/ingestion.py	Extracts document text
Chunking	app/chunking.py	Splits text and applies section labels

Component	File	Purpose
Embeddings	app/embeddings.py	Generates vector embeddings
Vector store	app/vector_store.py	FAISS / pgvector retrieval backend
RAG orchestration	app/rag.py	Builds answer and section references
Web UI	streamlit_app.py	User-facing interface

5. Test Types

5.1 Unit Testing

Validate individual functions/modules in isolation.

Focus areas:

- chunk creation
- section labeling
- deduplication logic
- vector store search behavior
- fallback answer formatting

5.2 Integration Testing

Validate that modules work together correctly.

Focus areas:

- upload → extract → chunk → embed → store flow
- ask → retrieve → answer flow
- FastAPI + vector store + database integration

5.3 System Testing

Validate the complete end-to-end workflow from the Streamlit UI and API.

5.4 Regression Testing

Ensure new changes do not break:

- pgvector support,
- section references,
- answer clarity,
- duplicate prevention.

5.5 Non-Functional Testing

- usability
- reliability
- basic performance
- graceful failure handling

6. Test Environment

Hardware / OS

- Windows workstation

Software

- Python 3.10+
- PostgreSQL with pgvector
- Virtual environment .venv
- Streamlit
- FastAPI / Uvicorn
- Pytest
- pytest-cov

Environment Variables

Expected configuration:

```
VECTOR_DB_BACKEND=pgvector
PGVECTOR_DSN=postgres://postgres:<password>@localhost:5432/ragdb
PGVECTOR_TABLE_NAME=rag_embeddings
PGVECTOR_PRIMARY_SEARCH=pgvector
LLM_PROVIDER=openai    # or huggingface
```

Test Commands

```
python -m pytest -q
uvicorn app.main:app --reload
streamlit run streamlit_app.py
```

Test Requirements Baseline

The minimum test requirements for this project are:

- the `tests/` suite must run from the repository root using the existing `pytest.ini` configuration,
- API regression tests must cover `/upload` and `/ask`,
- core pipeline tests must cover ingestion, retrieval, fallback behavior, and vector-store logic,
- UI validation must cover document upload, question submission, clear-history behavior, and API offline messaging,
- pgvector-specific checks are required when `VECTOR_DB_BACKEND=pgvector` or `hybrid`,
- each defect fix must add or update at least one relevant automated or manual regression test,
- release validation must include both automated evidence and a short manual QA summary.

Test Evidence Requirements

Each planned test cycle should produce the following evidence:

- console output from the executed `pytest` command,
- a pass/fail summary with executed scope,
- defect references for failed or blocked scenarios,
- coverage output for regression or release runs,
- manual QA notes for UI and environment-specific checks.

7. Entry Criteria

Testing may begin when:

- dependencies are installed,
- PostgreSQL is running,
- `ragdb` exists,
- `pgvector` extension is enabled,
- application launches successfully,
- test environment variables are configured,
- required sample documents and negative-test files are available,
- reporting output location for the current cycle is agreed.

8. Exit Criteria

Testing is considered complete when:

- all critical flows pass,
- no blocker or critical defects remain open,
- API upload/ask flow is verified,
- section references appear correctly in answers,
- regression suite passes,
- database persistence is confirmed,
- test results and supporting evidence are captured in the test report.

9. Test Data

Sample Documents

1. **Simple TXT** — short structured text with headings
2. **Long TXT/PDF** — multiple sections and repeated keywords
3. **DOCX sample** — formatted headings and paragraphs
4. **Unsupported file** — `.csv` or `.exe` for negative testing
5. **Empty document** — blank or near-empty content

Sample Questions

- "What is the document about?"
- "Summarize Section 2."
- "What are the benefits mentioned?"
- "Which section discusses requirements?"
- "Ask something not present in the document."

10. Test Scenarios and Cases

10.1 API Tests

ID	Test Case	Steps	Expected Result
API-01	Upload valid TXT	POST /upload with .txt file	200 OK , success message returned
API-02	Upload valid PDF	POST /upload with .pdf file	Text processed successfully
API-03	Upload valid DOCX	POST /upload with .docx file	Text processed successfully
API-04	Ask before upload	POST /ask without prior upload	Friendly message: no document uploaded
API-05	Ask valid question	Upload file, then POST /ask	Clear answer is returned
API-06	Verify section references	Ask document question	Answer includes References: section
API-07	Unsupported file type	Upload unsupported file	Error handled cleanly
API-08	Empty question	Send blank question payload	Validation or graceful failure

10.2 UI Tests

ID	Test Case	Steps	Expected Result
UI-01	Open Streamlit app	Launch browser to app URL	UI loads successfully
UI-02	Upload document through UI	Select file and click process	Success indicator shown
UI-03	Ask question through UI	Enter question and submit	Answer displayed in UI
UI-04	Clear history	Click clear button	Chat history removed
UI-05	API offline state	Stop API and refresh UI	Proper offline error shown
UI-06	File change resets stale state	Select file A, then select file B	Previous document and chat state is cleared before new processing
UI-07	Enter-to-submit question	Type question and press Enter	Same behavior as clicking submit

10.3 Chunking and Retrieval Tests

ID	Test Case	Steps	Expected Result
RET-01	Chunk long document	Process long document	Chunks created with overlap
RET-02	Section labeling	Process sectioned text	Chunks contain section labels
RET-03	Deduplicate retrieval results	Ask repeated-topic question	Repeated chunks are not returned redundantly
RET-04	Clear stale data on re-upload	Upload document A then B	Answers reflect latest document only

10.4 Database / pgvector Tests

ID	Test Case	Steps	Expected Result
DB-01	PostgreSQL connection	Start app with pgvector config	Connection succeeds
DB-02	Table initialization	Run setup SQL	<code>rag_embeddings</code> table exists
DB-03	Data insertion	Upload document	Rows inserted into <code>rag_embeddings</code>
DB-04	Data clearing on re-upload	Upload second document	Old rows cleared/replaced
DB-05	Query vector search	Ask question	Relevant rows retrieved from pgvector

10.5 LLM and Fallback Tests

ID	Test Case	Steps	Expected Result
LLM-01	Valid provider response	Use valid API key	Generated answer returned
LLM-02	Invalid API key	Use wrong key	App fails gracefully
LLM-03	Quota exceeded	Simulate provider quota issue	Clear fallback answer from document text
LLM-04	Question not answered by context	Ask unrelated question	Returns <code>I don't know.</code> or grounded fallback

10.6 Performance and Stability Tests

ID	Test Case	Steps	Expected Result
PERF-01	Upload medium file	Upload 1–5 MB file	Completes within acceptable time

ID	Test Case	Steps	Expected Result
PERF-02	Repeated queries	Ask multiple questions sequentially	Stable answers, no crash
PERF-03	Restart and reconnect	Restart backend/UI	App becomes available again

10.7 Usability Tests

ID	Test Case	Steps	Expected Result
USE-01	First-time upload flow clarity	Open UI without prior context and attempt upload	Labels and actions are understandable without extra guidance
USE-02	Question submission discoverability	Upload a document, then ask a question using button or Enter	User can identify how to submit a question without confusion
USE-03	Error message clarity	Trigger API offline or invalid-input state	Error message explains the problem and next action clearly
USE-04	History and clear action clarity	Ask multiple questions, then clear history	User can distinguish past answers and understands the effect of Clear
USE-05	File-change behavior clarity	Select a new file after interacting with a previous one	UI state reset is understandable and does not appear as data loss or malfunction
USE-06	Readability of answer output	Review returned answer and references	Answer text, references, and status messages are easy to scan

11. Expected Results Quality Criteria

A response is acceptable when it is:

- **clear** and directly answers the question,

- **grounded** in retrieved document content,
- **not repeated** or stale,
- includes **section references** like:

References: [Section 1: Introduction]

- does not expose raw stack traces to the end user.

A usability result is acceptable when:

- primary actions are easy to identify,
- form and button behavior are consistent,
- state changes are understandable,
- error messages tell the user what to do next,
- answer and reference text remains readable without extra explanation.

12. Defect Severity Guidelines

Severity	Description
Critical	App crash, API unusable, database inaccessible
High	Wrong answer source, repeated/stale results, upload failure
Medium	Missing references, unclear error messaging, UI inconsistency
Low	Cosmetic formatting issues, minor text layout problems

13. Risks and Mitigations

Risk	Impact	Mitigation
Invalid/OpenAI key	No generated answer	Use fallback answer path or valid provider key
PostgreSQL on wrong port	Missing data visibility	Standardize on localhost:5432

Risk	Impact	Mitigation
Duplicate chunks in DB	Repeated answers	Clear and deduplicate on upload/search
Large documents	Slow processing	Limit test sizes and monitor performance
Unsupported file contents	Extraction failure	Add negative/error-handling tests

14. Automation Strategy

Automated coverage should include:

- tests/test_api.py
- tests/test_rag_pipeline.py
- tests/test_vector_store.py
- tests/test_ingestion.py

Recommended CI gate:

```
python -m pytest -q
```

Recommended report-generation commands:

```
python -m pytest -q --junitxml=reports/test-results.xml
python -m pytest -q --cov=app --cov=tests --cov-report=term-missing --cov-report=xml:reports/coverage.xml
```

Recommended automation outputs:

- reports/test-results.xml for CI result publishing,
- reports/coverage.xml for machine-readable coverage tracking,
- reports/htmlcov/ for human-readable coverage review,
- console logs for fast troubleshooting.

Future improvements:

- add UI automation for Streamlit,
- add performance benchmarks,
- add database fixture setup/teardown.

15. Traceability Matrix

Requirement	Covered By
Upload supported documents	API-01, API-02, API-03, UI-02
Ask grounded questions	API-05, UI-03, LLM-01
Section-based references	API-06, RET-02
No repeated answers	RET-03, RET-04
PostgreSQL persistence	DB-01 to DB-05
Graceful provider failure	LLM-02, LLM-03
Usable UI behavior	UI-01 to UI-07, USE-01 to USE-06

16. Test Reporting Plan

16.1 Report Types

- **Developer smoke report:** short local summary for targeted changes before review.
- **Regression report:** CI-backed summary for the automated suite.
- **Release readiness report:** combined automated and manual QA summary used for sign-off.

16.2 Required Report Contents

Each formal test report should include:

- test cycle name, date, owner, and environment,
- build or commit reference,
- executed scope and omitted scope,
- pass/fail/blocked counts,
- linked defect IDs or issue references,
- key quality risks or known gaps,
- coverage summary when a regression or release run is performed,
- final recommendation: pass, pass with risk, fail, or blocked.

16.3 Reporting Cadence

- For local feature work: record a smoke result before requesting review.
- For CI runs: publish an automated regression result on each merge-relevant run.
- For release candidates: publish a release readiness report after manual UI and environment validation.

16.4 Report Ownership

- Developers own local smoke summaries for their changes.
- QA or the feature owner owns the regression summary.
- The release approver owns the final release readiness decision.

16.5 Report Artifacts

Preferred report artifacts are:

- terminal output from `pytest`,
- JUnit XML test results,
- coverage XML and HTML outputs,
- manual UI checklist notes,
- usability observation notes with issues and recommendations,
- database verification notes for pgvector-backed runs.

16.6 Status Definitions

Status	Meaning
Pass	All required tests passed with no blocking risk
Pass with Risk	Tests passed but known non-blocking issues remain documented
Fail	One or more required tests failed or a blocker remains open
Blocked	Testing could not complete because an environment or dependency issue prevented execution

17. Sign-off Criteria

The release is recommended when:

- all critical and high-priority tests pass,
- automated tests pass in CI,
- the Streamlit UI and FastAPI API are operational,
- answers are concise and section-referenced,
- PostgreSQL/pgvector data is visible and correct,
- a release readiness test report has been reviewed and accepted.

18. Summary

This test plan ensures the RAG application is validated across:

- **functionality,**
- **data retrieval quality,**
- **answer clarity,**
- **database correctness,**
- **fallback behavior,**
- **UI and API usability.**

It is intended to support both manual QA and automated regression testing for ongoing development.